

HERMES - High-frequency Emergency and Rural Multimedia Exchange System Software Description

Rhizomatica

February 21, 2026

Abstract

This document describes the software stack of the HERMES (High-frequency Emergency and Rural Multimedia Exchange System) - a digital communication system for the HF band which uses a high-performance HF modem and UUCP networking (Unix to Unix Communication Protocol), for email transport and other ad-hoc services. HERMES is also composed of email software infrastructure and Web Interface for access to the services, which run on a computer.

Contents

1	Introduction	2
2	HERMES Description	3
3	HERMES System Architecture	4
3.1	Topology and station roles	4
3.2	Reference architecture (v2 / sBitx + VARA-compatible TNC + UUCP)	4
3.3	Station component diagram	5
3.4	Service orchestration (systemd)	5
3.5	Interfaces: ports, paths and IPC	6
3.6	Dynamic behavior (message and call flows)	7
3.6.1	Outbound session (gateway calls a remote station)	7
3.6.2	Inbound session (remote answers an incoming call)	7
3.7	Variants	7
3.8	Architecture reference sources	7
4	HERMES Software Stack	8
4.1	ALSA (Audio configuration)	8
4.2	Radio control: trx_v2-userland (sbitx_controller)	9
4.2.1	sbitx.service (systemd unit)	9
4.2.2	Configuration files (/etc/sbitx)	9
4.2.3	Shared memory control API (sbitx_client, uucpd)	9
4.2.4	Websocket status API (optional)	9
4.2.5	Health checks	10
4.3	Modem configuration	10
4.3.1	VARA	10
4.3.2	Open modem: hermes-modem (“mercury” binary)	11
4.3.3	ARDOP	11
4.3.4	Mercury (R&D note)	12
4.4	UUCP (Network configuration)	12
4.5	UUCPD: uucpd + uuport (UUCP to modem bridge)	13
4.5.1	Install	13
4.5.2	UUCP configuration (/etc/uucp)	13
4.5.3	uucpd.service (systemd unit)	14
4.5.4	Health checks	14
4.5.5	Running uucpd (examples)	14
4.5.6	Command-line usage (uucpd / uuport)	15
4.6	User Web Interface	15
4.6.1	Hermes-GUI	15
4.6.2	Hermes-API	16

1 Introduction

Telecommunication in the HF Band using skywave propagation offers hundreds to thousands of kilometers coverage with relative low power and simple antennas, but it is also challenging, given the extreme limitation on throughput and very long transceiver turn-around times between tx and rx.

HERMES is a digital telecommunication system for the HF band, which uses a high-performance HF modem and UUCP (Unix-to-Unix Communication Protocol), for email transport and other ad-hoc services. UUCP is arguably the most relevant store-and-forward system to date, in use since the late 70’s for software exchange, email, bulletin board systems news and even military communication. First released in Bell Labs’ Unix Version 7, it is still present in all major Unixes (including Linux) and integrate seamless with email software like Postfix.

Upper network layers are composed by email software infrastructure and Web Interface for management and access to the services. HERMES network topology is flexible, but typically configured as star, with a “gateway” station located where Internet is available (in order to route data from stations located in isolated areas) and “remote” stations located at communities without access to other telecommunication systems.

2 HERMES Description

This section briefly introduces what HERMES system can do today. HERMES is basically composed of:

- HF transceiver and associated antenna, connected to a computer;
- Computer running the HERMES free software stack;
- The computer provides local connectivity through WiFi (or other local-area network technology)

Rhizomatica developed the HERMES hardware which embeds all the needed parts for a simple to use HF telecommunication equipment. Figure 1 and 2 contains a picture of the HERMES box. The HERMES box contains everything needed for joining a HERMES network, while users have access to the HERMES services over WiFi or ethernet.



Figure 1: The HERMES hardware at a “gateway” location, a power supply (yellow), a standard HF voice radio (top right) and a wattmeter (top left).

HERMES uses GNU UUCP with Debian downstream patches (version 1.07-27 or greater, available at: <https://packages.debian.org/bullseye/uucp>).

Specific HERMES network stack source code, radio firmware and other system components are available at:

- Main network components and firmware: <https://github.com/Rhizomatica/hermes-net>
- REST API: <https://github.com/Rhizomatica/hermes-api>
- Web GUI: <https://github.com/Rhizomatica/hermes-gui>
- Documentation: <https://github.com/Rhizomatica/hermes-documentation>



Figure 2: The HERMES hardware at a “remote” location, with a battery appearing the bottom left.

3 HERMES System Architecture

This section provides an architecture-level description of HERMES as implemented by the software components referenced in Section 3.8. The focus is the HF data plane (store-and-forward payload transport over HF) and the radio/modem control plane that makes that possible on commodity HF transceivers.

3.1 Topology and station roles

HERMES is commonly deployed as a star topology network with two roles:

- **Gateway station:** has Internet access and orchestrates synchronization with remote stations.
- **Remote station:** provides local services (WiFi + web UI + email) and exchanges data with the gateway over HF.

Inter-station exchange is **asynchronous** (store-and-forward): payloads are queued locally and later transported over an HF link during scheduled communication windows (often initiated by the gateway in round-robin fashion).

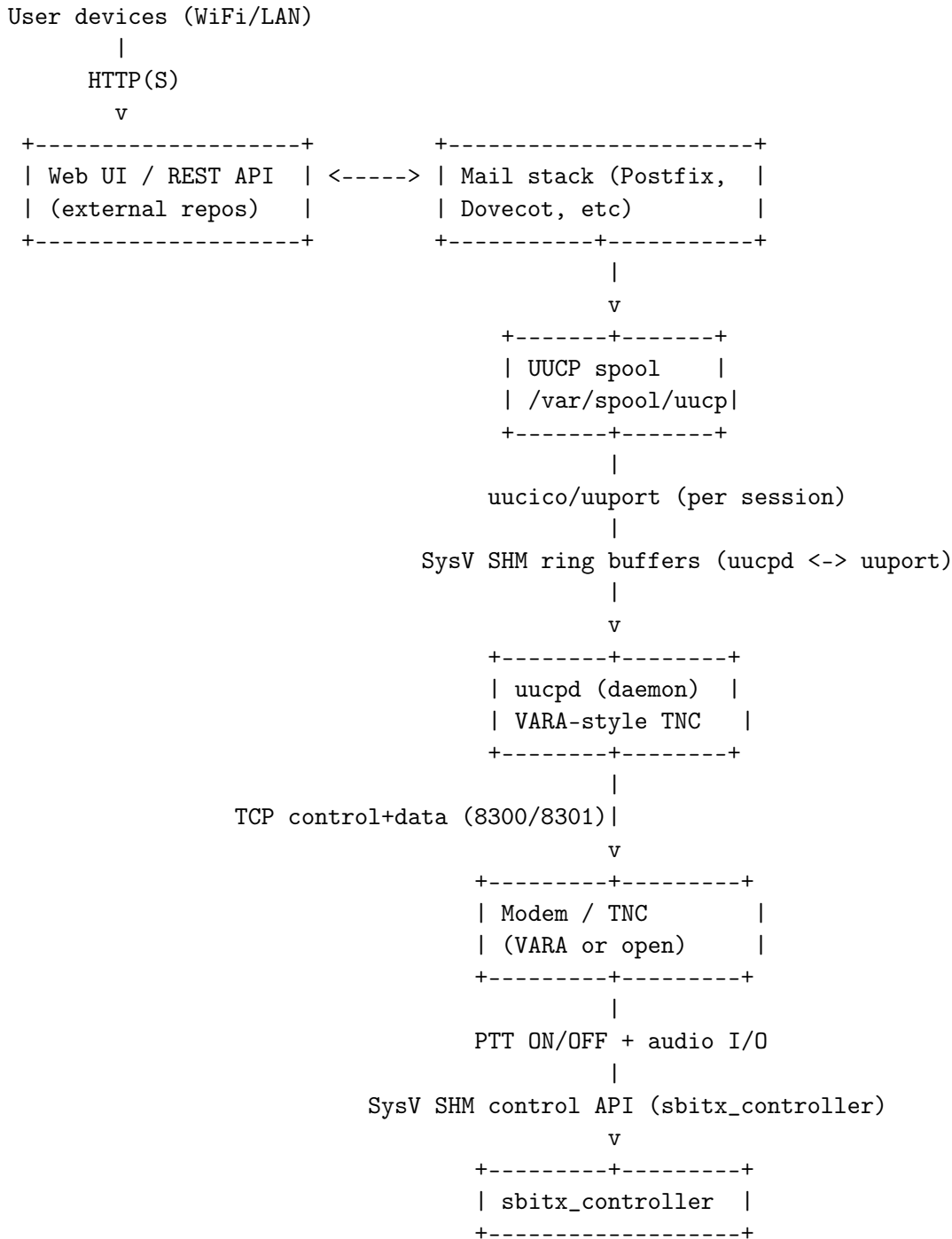
3.2 Reference architecture (v2 / sBitx + VARA-compatible TNC + UUCP)

The reference implementation described in this document corresponds to HERMES v2 (sBitx-based radio control) and a **VARA-compatible TCP TNC interface** on localhost. In this model the station is composed of the following main processes:

- **Radio control daemon:** `sbitx_controller` (from `hermes-net/trx_v2-userland`), started by systemd as `sbitx.service`. It controls RF hardware and exposes a shared-memory control/status interface used by other services.
- **Modem/TNC:** either **VARA HF** (proprietary) or an **open modem** that implements the same VARA-style TCP TNC control/data sockets (e.g. `hermes-modem`). The control socket is typically on TCP 8300 and the data socket on 8301 (base port + 1).
- **UUCP transport:** GNU UUCP (`uucico` + spool under `/var/spool/uucp`), which provides store-and-forward job management and email transport integration.

- **UUCP-to-modem bridge:** `uucpd` (daemon) + `uuport` (pipe helper) from `hermes-net/uucpd`. This is the key component that turns the modem/TNC into an UUCP “port”.
- **Gateway orchestration (gateway-only):** a scheduler such as `caller.sh` (installed as `caller.service`) that triggers `uucico -S <station>` calls according to a schedule and station list.

3.3 Station component diagram



3.4 Service orchestration (systemd)

On Debian-based HERMES images these components are typically started by `systemd`. Reference unit files are shipped in `hermes-net/system_services/init/`. Some helper units for the Wine-based VARA modem are shipped by `hermes-installer` under `hermes-installer/conf/x11vnc/`. The most relevant ones for the HF data plane are:

```

sbitx.service
    ExecStart=/usr/bin/sbitx_controller

# VARA HF (Wine + GUI): headless X server + VNC helper units
x11.service
    ExecStart=xvfb-run ... -n 69 -l xstartup

vnc.service
    ExecStart=x11vnc -display WAIT:69 ...

uucpd.service
    ExecStart=/usr/bin/uucpd -a 127.0.0.1 -p 8300 -r vara -o shm -f 2750p -m

caller.service    (gateway-only scheduler)
    ExecStart=/usr/bin/caller.sh

# Open modem (hermes-modem) alternative:
modem.service
    ExecStart=/usr/bin/mercury -v

uucpd-mercury.service    (typically installed as uucpd.service)
    ExecStart=/usr/bin/uucpd -a 127.0.0.1 -p 8300 -r vara -o shm -m -F

```

3.5 Interfaces: ports, paths and IPC

The HERMES reference architecture uses a small set of explicit interfaces. The most important ones are summarized below.

Layer	Interface	Notes
Radio control	<code>/etc/sbitx/core.ini</code> , <code>/etc/sbitx/user.ini</code>	Installed by hermes-net ; defines radio IO, profiles, etc.
Radio control IPC	SysV SHM key 66650 (<code>controller_conn</code>)	Command/response API used by sbitx_client and by uucpd for PTT/LED/status.
Modem/TNC	TCP <code>base_port</code> and <code>base_port+1</code> (default 8300/8301)	VARA-style TNC interface: control socket (ASCII commands) + data socket (raw stream).
UUCP config	<code>/etc/uucp/{config,sys,port,...}</code>	<code>config</code> provides <code>nodename</code> ; <code>sys</code> defines peers and schedules; <code>port</code> uses <code>type pipe</code> .
UUCP spool	<code>/var/spool/uucp</code>	Store-and-forward queue; inspected with uustat and logs with uulog .
UUCP bridge IPC	SysV SHM keys 66664/66666/66668	Shared connector struct + in/out ring buffers used by uucpd and uuport .

3.6 Dynamic behavior (message and call flows)

3.6.1 Outbound session (gateway calls a remote station)

1. Payloads (emails/messages/files) are queued locally and end up as UUCP jobs in the UUCP spool.
2. The gateway scheduler (e.g. `caller.sh`) triggers an UUCP session: `uucico -S <remote>`.
3. `uucico` starts in master mode and invokes `uuport` (configured as a `type pipe` UUCP port).
4. `uuport` attaches to `uucpd`'s shared memory buffers and becomes `uucico`'s byte-stream "modem".
5. `uucpd` maintains a persistent TCP connection to the modem/TNC. When it sees outgoing bytes and is not connected, it issues a `CONNECT <local> <remote>` command on the TNC control socket.
6. The modem/TNC drives the RF transmit/receive turn-taking and signals `PTT ON`/`PTT OFF` on the control socket. `uucpd` translates these into actual radio keying (serial CAT or shared-memory keying via `sbitx_controller`).
7. Once connected, bytes flow bidirectionally: `uucico` ↔ `uuport` ↔ `uucpd` ↔ modem/TNC ↔ HF RF link.

3.6.2 Inbound session (remote answers an incoming call)

1. The modem/TNC is kept in listen mode by `uucpd` (`LISTEN ON`).

2. When a remote station calls in, the modem/TNC reports **CONNECTED** ... on the control socket.
3. **uucpd** detects this as an inbound connection and forks `/usr/sbin/uucico` in slave mode, wiring its stdin/stdout to the same shared buffers used by **uuport**.
4. The UUCP protocol runs and the spool is updated (receive jobs delivered; pending jobs may be sent back).

3.7 Variants

- **Open modem (hermes-modem)**: provides a VARA-style TCP TNC interface, so from **uucpd** perspective it behaves like VARA and uses the same integration path (control/data sockets, PTT ON/OFF messages).
- **ARDOP (legacy)**: supported by **uucpd** using its `-r ardop` mode; kept mainly for compatibility with older deployments.
- **HERMES v1 (uBitx)**: legacy radio control uses `ubitx_controller` and different service units (see `hermes-net/system_services/init`).

3.8 Architecture reference sources

The architecture above is grounded in the following source trees and artifacts:

- `hermes-net/uucpd` (bridge daemon **uucpd** + port helper **uuport** + modem integration code)
- `hermes-net/trx_v2-userland` (`sbitx_controller` shared-memory control API)
- `hermes-net/system_services/init` (reference systemd unit files such as `sbitx.service`, `uucpd.service`, `caller.service`)
- `hermes-modem` (open modem implementing VARA-style TCP TNC)

4 HERMES Software Stack

This section describes the main software components that implement the architecture described in Section 3.2.

HERMES deployments commonly use **VARA HF** as modem/TNC, but the system architecture is intentionally based on a VARA-style TCP TNC interface, allowing **open modem** implementations (such as **hermes-modem**) to be used as a drop-in replacement. ARDOP is supported mostly for legacy compatibility.

For the network layer, UUCP is used. UUCP is a system for asynchronous store-and-forward communication first released in Bell Labs Unix V7 in the late 1970s, and still used today in niches, like communication over HF.

The **uucpd** project (historically also referred to as **uuardopd** / **rhizo-uuardop** in older documents and code comments) was developed to integrate UUCP with HF modems/TNCs.

At a high level, the HERMES station stack is made of:

- Radio control (`sbitx_controller` and `sbitx_client`)
- Audio subsystem configuration (ALSA and related device plumbing)
- Modem/TNC (VARA HF or open modem with VARA-style TCP interface)
- UUCP (spool and `uucico` sessions)
- **uucpd** bridge (**uucpd** daemon + **uuport** helper)
- Upper services (web UI/API + email infrastructure)

4.1 ALSA (Audio configuration)

HERMES relies on a modem/TNC that exchanges baseband audio with the HF transceiver. On Linux, ALSA configuration is commonly used to provide stable capture/playback device mapping for the modem process (including Wine-based VARA deployments).

The reference configuration shipped in `hermes-net` is installed as `/etc/asound.conf` from `hermes-net/system_`

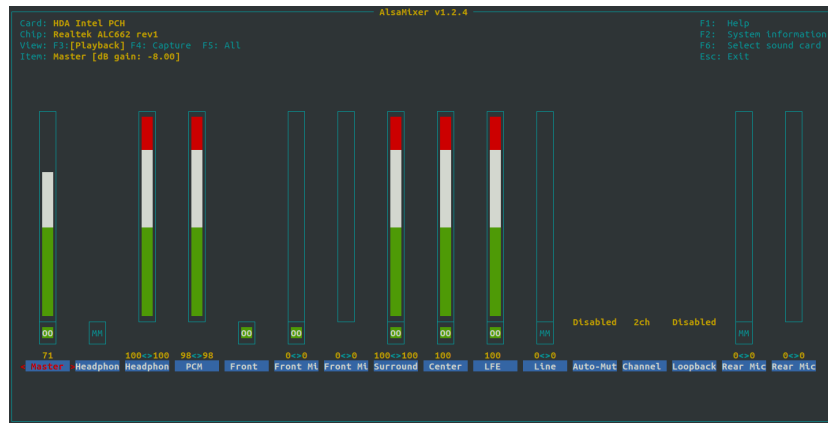
```
pcm.asymwine {
    type asym
    capture.pcm "hw:1,1"
    playback.pcm "hw:2,0"
}

pcm.!default {
    type plug
    slave.pcm "asymwine"
}

pcm.dsp {
    type plug
    slave.pcm "asymwine"
}
```

The actual `hw:X,Y` indices depend on your hardware. To identify them, use for example:

```
alsamixer
arecord -l
aplay -l
```



4.2 Radio control: `trx_v2-userland` (`sbitx_controller`)

HERMES v2 uses Rhizomatica's sBitx-based radio control stack from `hermes-net/trx_v2-userland`. It builds two main executables:

- **sbitx_controller**: a daemon that controls the radio (RF control + GPIO + I2C devices + audio DSP).
- **sbitx_client**: a CLI tool used for monitoring and setting radio parameters (frequency, mode, power, etc.).

4.2.1 sbitx.service (systemd unit)

The controller is typically started by systemd as `sbitx.service`:

```
hermes-net/system_services/init/sbitx.service
ExecStart=/usr/bin/sbitx_controller
```

4.2.2 Configuration files (/etc/sbitx)

The default configuration files are expected at:

- `/etc/sbitx/core.ini`
- `/etc/sbitx/user.ini`

They are installed by `hermes-net` (`make install`) from the `trx_v2-userland/config/` directory.

Core switches. The most relevant toggles in `core.ini` are:

- `enable_shm_control = 1` (required for `sbitx_client` and for `uucpd -o shm` integration);
- `enable_websocket = 1` (optional; used by web GUIs for status streaming).

4.2.3 Shared memory control API (sbitx_client, uucpd)

The controller exposes a System V shared memory segment (key 66650, type `controller_conn`, see `hermes-net/include/sbitx_io.h`). This interface is used by `sbitx_client` and also by `uucpd` when configured with `-o shm`, allowing the modem/TNC integration to key PTT (and report status such as bitrate/SNR/bytes) without direct serial/CAT access.

4.2.4 Websocket status API (optional)

When `enable_websocket = 1`, `sbitx_controller` starts an HTTPS/WSS server (Mongoose-based) that:

- listens on `wss://0.0.0.0:8080`;
- upgrades `/websocket` to a websocket stream (JSON status updates);
- serves static files from `/etc/sbitx/web` (installed by `hermes-net` from `trx_v2-userland/web/`).

For example, the test page is typically available at `https://<station>:8080/test.html` and connects to `wss://<station>:8080/websocket`.

TLS files are expected at:

- `/etc/ssl/certs/hermes.radio.crt`
- `/etc/ssl/private/hermes.radio.key`

4.2.5 Health checks

```
systemctl status sbitx
sbitx_client -c get_frequency -p 0
sbitx_client -c get_mode -p 0
```

If websocket is enabled:

```
ss -lntp | grep 8080
journalctl -f -u sbitx
```

4.3 Modem configuration

4.3.1 VARA

VARA is a High Performance HF modem based on OFDM modulation. VARA modem is the only piece of private software in the stack.

Currently, HERMES uses VARA HF 4.3 or newer as it's standard modem. Use this protocol with your HERMES if you are looking for best performance available.

Please refer to the developer's website for download and instructions on installation and use:

<https://rosmodem.wordpress.com/>

Configuration

Edit `/opt/VARA/VARA.ini.default` to include your license key and callsign.

x11.service + vnc.service (headless VARA). In the reference Debian images, VARA HF is run under Wine and requires an X server. For headless stations, **hermes-installer** ships a minimal Xvfb + x11vnc setup:

```
hermes-installer/conf/x11vnc/x11.service
```

```
ExecStart=xvfb-run --server-args="-screen 0 1024x768x24" ... -n 69 -l xstartup
```

```
hermes-installer/conf/x11vnc/vnc.service
```

```
ExecStart=x11vnc -display WAIT:69 ...
```

The `xstartup` script (`hermes-installer/conf/xstartup`) launches `wine /opt/VARA/VARA.exe` (via `xterm`) and starts the window manager used by this session.

Health checks.

```
systemctl status x11 vnc
```

```
journalctl -f -u x11
```

```
journalctl -f -u vnc
```

```
# VARA-style TNC should listen on the TCP base port (default 8300):
```

```
ss -lntp | grep 8300
```

Integration contract (VARA-style TCP TNC). The rest of the HERMES HF stack does not interact with VARA through audio directly. Instead, `uucpd` connects to VARA's TCP interface (control socket on `base_port`, data socket on `base_port+1`; typically 8300/8301). VARA signals radio keying by sending PTT ON/PTT OFF on the control socket; `uucpd` translates that into actual PTT via serial or shared-memory radio control (`-o shm` for sBitx).

4.3.2 Open modem: hermes-modem ("mercury" binary)

Rhizomatica's open modem implementation is available at: <https://github.com/Rhizomatica/hermes-modem/>

It provides a VARA-style TCP TNC interface (separate control and data sockets) so it can be used with the same `uucpd` integration path described above (`uucpd -r vara ...`). In addition to ARQ links, it also provides a dedicated TCP port for broadcast data mode (default 8100).

The **hermes-modem** README documents its CLI options. In summary:

- `-p` sets the ARQ TCP base port (control `base`, data `base+1`; default 8300).
- `-b` sets the broadcast TCP port (default 8100).
- `-x` selects the audio backend (`alsa`, `pulse`, `shm`, etc.).

modem.service (systemd unit). A reference unit file is shipped as `modem.service` (see `hermes-net/system_se`). It starts the modem directly via `/usr/bin/mercury` (installed from `hermes-modem` using `make install`). On Linux, `mercury` defaults to ALSA and uses the ALSA device `default` for capture/playback unless `-i/-o` are provided. When used, the corresponding `uucpd-mercury.service` unit depends on `modem.service`.

Health checks.

```
systemctl status modem
journalctl -f -u modem
ss -lntp | grep 8300
ss -lntp | grep 8100
```

4.3.3 ARDOP

ARDOP is a SDR(software defined radio) modem compatible with both HF and VHF radio transmissions. In HERMES it is mostly a legacy/optional choice; the preferred integration target is the VARA-style TCP TNC interface described above. The protocol can operate over a wide range of data rates by automatically adjusting for optimized performance under poor multi-path conditions. Use this protocol with your HERMES if you are looking for a completely open-source and free option.

Download link: <https://github.com/DigitalHERMES/ardopc>

Configuration

The ardop binary should be in `/usr/bin/ardop`, which can be a symbolic link to `/usr/bin/ardop1ofdm`, `ardop2`, `ardopofdm`.

ARDOP service file for the ICOM IC-7100 (USB connection, PTT done over serial):

```
[Unit]
Description=ARDOP daemon

[Service]
Type=simple
ExecStart=/usr/bin/ardop 8515 -c /dev/ttyUSB0 ARDOP ARDOP -k FEFE88E01C0001FD -u FEFE88E01C0000FD
ExecStop=/usr/bin/killall -s QUIT ardop
IgnoreSIGPIPE=no
#StandardOutput=null
#StandardError=null
StandardOutput=syslog
StandardError=syslog

[Install]
WantedBy=multi-user.target
```

Service file when using a VOX based setup (eg. when using an interface like the Signalink):

```
[Unit]
Description=ARDOP daemon

[Service]
Type=simple
ExecStart=/usr/bin/ardop 8515 ARDOP ARDOP
ExecStop=/usr/bin/killall -s QUIT ardop
IgnoreSIGPIPE=no
```

```
#StandardOutput=null
#StandardError=null
StandardOutput=syslog
StandardError=syslog

[Install]
WantedBy=multi-user.target
```

Start/stop ARDOP service:

```
systemctl start ardop.service
systemctl stop ardop.service
```

See the log:

```
journalctl -f -u ardop
```

4.3.4 Mercury (R&D note)

The name “Mercury” is used historically in HERMES for ongoing modem R&D work. In practice, the current open modem implementation used in HERMES deployments is **hermes-modem** (previous subsection), which exposes the VARA-style TCP TNC interface expected by **uucpd**. The standalone Mercury R&D repository (when applicable) is: <https://github.com/Rhizomatica/mercury/>

4.4 UUCP (Network configuration)

UUCP is an acronym of Unix-to-Unix Copy. It’s a protocol for remote execution of commands and transfer of files, email and netnews between computers.

Since HERMES uses Debian GNU/Linux release Bullseye, UUCP packages are installed by default.

If you are not using Debian Bullseye, UUCP Debian package version 1.07-27 or higher should be used, for example, the version from Debian Bullseye (11): <https://packages.debian.org/bullseye/uucp>

UUCP command line examples follow. To copy a file to a remote host, the following command adds a copy job to the uucp queue (“-r” is used to not start transmission after queuing):

```
uucp -C -r -d source.xxx AM4AAB\!/var/www/html/arquivos/${nodename}/
```

Trigger the transmission of all queued jobs for host AM4AAA:

```
uucico -S AM4AAA
```

List the jobs:

```
uustat -a
```

Kill a job:

```
uustat -k job
uustat -K
```

See the log:

```
uulog
```

4.5 UUCPD: uucpd + uuport (UUCP to modem bridge)

UUCPD is a set of tools that allow UUCP to operate over HF links by integrating `uucico` with an HF modem/TNC. In the HERMES v2 reference architecture (Section 3.2) the modem/TNC is accessed through a VARA-style TCP interface (control socket on `base_port`, data socket on `base_port+1`).

The `hermes-net/uucpd` package provides two executables:

- `uucpd`: the always-on bridge daemon that keeps the modem/TNC in listen mode, initiates outgoing connects when data is available, and starts `uucico` in slave mode when an inbound call is received.
- `uuport`: the helper invoked by `uucico` when calling out (UUCP port type `pipe`). It attaches to `uucpd`'s shared memory buffers and provides a byte-stream interface to `uucico`.

Communication between `uuport` and `uucpd` uses System V shared memory (connector struct + ring buffers). When configured with `uucpd -o shm`, radio keying/status is also done through the `sbitx_controller` shared-memory interface (Section 4.2).

Download link: <https://github.com/Rhizomatica/hermes-net/tree/main/uucpd>

4.5.1 Install

To compile and install, type:

```
$ make
$ make install
```

4.5.2 UUCP configuration (/etc/uucp)

Port configuration example at `"/etc/uucp/port"`:

```
port HFP
type pipe
command /usr/bin/uuport
```

An alternative Port configuration if you use a patched `uucp` (for `"\Z"` support, available in `"improved-pipe.patch"` which was added to `uucp` debian package version 1.07-27), where `uuport` passes the callsign of the station to be called to `uucpd` with the `uucp` remote station name (allowing a single `uucpd` instance to be used for different remote station callsigns):

```
port HFP
type pipe
command /usr/bin/uuport -c \Z
```

Sys protocol example (tested and works fine) at `"/etc/uucp/sys"`:

```
protocol y
protocol-parameter y packet-size 512
protocol-parameter y timeout 540
chat-timeout 200
```

Sys configuration example of remote system at `"/etc/uucp/sys"` (without login prompt):

```
system remote
call-login *
call-password *
time any
port HFP
chat "" \r
```

Sys configuration example of remote system at `"/etc/uucp/sys"` (with login prompt - should call `uucpd` with `"-l"`):

```
system remote
call-login *
call-password *
time any
port HFP
chat "" \r\c ogin: \L word: \P
```

4.5.3 uucpd.service (systemd unit)

The reference systemd unit files shipped with `hermes-net` include:

- `hermes-net/system_services/init/sbitx.service`
- `hermes-net/system_services/init/uucpd.service`
- `hermes-net/system_services/init/caller.service` (gateway-only)

In the reference VARA deployment (Wine VARA + VNC), the `uucpd.service` unit is:

```
After=vnc.service sbitx.service
ExecStart=/usr/bin/uucpd -a 127.0.0.1 -p 8300 -r vara -o shm -f 2750p -m
```

In the open-modem (mercury) deployment, `hermes-net` ships `uucpd-mercury.service` and installs it as `uucpd.service`:

```
After=modem.service sbitx.service
ExecStart=/usr/bin/uucpd -a 127.0.0.1 -p 8300 -r vara -o shm -m -F
```

4.5.4 Health checks

```
systemctl status uucpd
systemctl start uucpd.service
systemctl stop uucpd.service
journalctl -f -u uucpd
```

```
# TNC base port (control) should be up (default 8300):
ss -lntp | grep 8300
```

4.5.5 Running uucpd (examples)

Examples of `uucpd` invocation:

```
$ uucpd -a 127.0.0.1 -c PU2BBB -p 8515 -t 60 -r ardop
$ uucpd -a 127.0.0.1 -p 8300 -r vara -o icom -s /dev/ttyUSB0 -f 2750p
$ uucpd -a 127.0.0.1 -p 8300 -r vara -o shm -f 2750p -m
```

4.5.6 Command-line usage (uucpd / uuport)

<code>-c callsign:</code>	Station Callsign (Eg: PU2HFF)
<code>-d remote_callsign:</code>	Remote Station Callsign (optional)
<code>-r [ardop,vara]:</code>	Choose modem/radio type
<code>-a tnc_ip_address:</code>	IP address of the TNC
<code>-p tcp_base_port:</code>	TCP base port of the TNC. ARDOP uses ports <code>tcp_base_port</code> and <code>tcp_base_port+1</code>
<code>-t timeout:</code>	Time to wait before disconnect when idling (ARDOP ONLY)
<code>-f features:</code>	Enable/Disable features.

Supported features ARDOP: ofdm, noofdm (default: ofdm)
 Supported features VARA, BW mode: 500, 2300 or 2750 (default: 2300)
 Supported features VARA, P2P mode: "p" to enable (eg. 2300p)
 -s serial_device: Set the serial device file path for keying the radio (VARA ONLY)
 -l: Tell UUCICO to ask login prompt (default: disabled)
 -F: Enable UUCP handshake fast-track (requires both ends; no-login chat)
 -m: Enable shared memory messaging parameter to uucp (requires patched uucp)
 -o [icom,ubitx,shm]: Sets radio type (supported: icom, ubitx or shm).
 -h: Prints this help

UUPORT Usage

-c system_name: Name of the remote system (default is don't change).
 -e logfile.txt: Log file (default is stderr).
 -h: Prints this help

4.6 User Web Interface

HERMES provides a web-based interface for sending messages, files and emails. It also allows for user management and radio configuration, among other features. Current implementation also supports symmetric cryptography and audio/image compression.

The web app is composed of the front-end interface and a REST API.

4.6.1 Hermes-GUI

Hermes-GUI is the front-end interface built with Angular.js.

Source code is located at: (<https://github.com/Rhizomatica/hermes-gui/>).

The project was generated with Angular CLI version 13.2.6. Angular: 13.2.6 Angular CLI: 13.2.6
 Node: 12.22.5 Package Manager: npm 8.5.2

Requirements of the user interface:

- ImageMagic: for image manipulation
- VVC: Best free software low bitrate visual content compression codec: <https://github.com/fraunhoferhhi/>
- LPCNet: Best free software audio compression codec: <https://github.com/xiph/LPCNet>
- GnuPG: For cryptography
- hostapd: WiFi AP mode software

The Web interface can be accessed by typing any address in a browser connected to the WiFi (set the DNS accordingly) or simply 10.0.0.1.

Install node, npm

Install node preferably(V12.22.5) and npm (node package manager) in your distro

Run 'npm install' inside the project path

Development server

Configure .env file with your parameters and run npx ts-node setEnv.ts to set .env values in environment.ts.

Run ng serve --configuration=en for a dev server in english, you can change the language to spanish (ng serve --configuration=es) or portuguese (ng serve --configuration=pt) if you wish. Navigate to

`http://localhost:4200/`. The app will automatically reload if you change any of the source files.

Code scaffolding

Run `ng generate component component-name` to generate a new component. You can also use `ng generate directive|pipe|service|class|guard|interface|enum|module`.

Running unit tests

Run `ng test` to execute the unit tests via Karma.

Running end-to-end tests

Run `ng e2e` to execute the end-to-end tests via Protractor.

Further help

To get more help on the Angular CLI use `ng help` or go check out the Angular CLI README.

Interface Contents

Inside the folder `src` you will find the angular templates for the system's interface.

On `style.less` there are the styling code, written in less.

Inside the `app` folder, you will find the template components. The `app-routing module.ts` file is responsible for linking the templates to url adressess. and the `app.component`, the application root template.

For every system component, there will be a folder with the `component.html`, `component.less`, and `component.ts`. On the `.html` file there is the html angular template, and on the `component.ts` you will find typescript code related to that component.

Also inside the `app` folder, there is the `_services` folder, where you can find common services shared between the components to access the [station-api] (<https://github.com/DigitalHERMES/station-api>).

On `assets` folder you find links to svgs and image files used on the interface design.

On the `locale` folder there are the `xlif` files for translation. For the translations, we are using `i18n` angular module. To generate translations, you need to use `ng extract-i18n --output-path src/locale/` to generate the `messages.xlif` file and then `xliffmerge --profile xliffmerge.json pt es` to transpose the new data to both `messages.es.xlif` and `messages.pt.xlif`, where you can input the new translation tokens.

4.6.2 Hermes-API

Hermes-API is a REST api for use on Hermes stations to exchange messages between them. It uses Lumen PHP Framework and composer to manage its own dependencies.

Source code is located at: (<https://github.com/Rhizomatica/hermes-api/>).

Server Requirements:

- Web server
- PHP >= 7.3
- OpenSSL PHP Extension
- PDO PHP Extension
- Mbstring PHP Extension
- SQLite DataBase

Installation settings

Setup your installation settings creating a .env file from .env.example

Run Composer

Composer is a dependency manager for PHP.

Run composer in your folder to install the dependencies and update relevant packages:

```
COMPOSER_ALLOW_SUPERUSER=1 composer install
COMPOSER_ALLOW_SUPERUSER=1 composer update
```

Copy hermes-api to /var/www/station-api/

Go to your hermes-api folder and run:

```
cp -rT . /var/www/station-api/
```

GNU Privacy Guard

Create the folder for GNU Privacy Guard and change owner:

```
mkdir -p /var/www/.gnupg
chmod 700 /var/www/.gnupg
chown www-data:www-data /var/www/.gnupg
```

Setup public folders for UUCP

Create and give permissions to inbox/outbox, downloads and temp folders to uucp public:

```
mkdir -p /var/www/station-api/storage/app/inbox
chmod 777 /var/www/station-api/storage/app/inbox
mkdir -p /var/www/station-api/storage/app/outbox
chmod 777 /var/www/station-api/storage/app/outbox
mkdir -p /var/www/station-api/storage/app/downloads
chmod 777 /var/www/station-api/storage/app/downloads
mkdir -p /var/www/station-api/storage/app/tmp
chmod 777 /var/www/station-api/storage/app/tmp
mkdir -p /var/www/station-api/storage/app/uploads
chmod 777 /var/www/station-api/storage/app/uploads
```

The structure created is used by the app as follows: /uploads (Files of outgoing messages) /downloads (Files generated from the inbox received messages) /inbox (incoming hermes message packs) /outbox (hermes message pack for deliver) /tmp (tmp files)

Link folders

```
ln -sf /var/www/station-api/public/ /var/www/html/api
```

Change owner of the app folder

```
chown -R www-data:www-data /var/www/station-api/
```

create database.sqlite file in database folder

To start a fresh database: php artisan migrate:refresh --seed

Database setup

Create a database named hermes and user with all privileges:

```
echo "CREATE USER hermes IDENTIFIED BY 'db_hermes'; " > sql_commands.sql
echo "CREATE DATABASE hermes;" >> sql_commands.sql
echo "GRANT ALL PRIVILEGES ON hermes.* TO hermes;" >> sql_commands.sql
```

```
mysql < sql_commands.sql
```

Then go to your hermes-api folder and run Artisan for the migrations:

```
cd /var/www/station-api/
php artisan migrate
php artisan db:seed
```

Running on port 8000:

```
> php -S localhost:8000 -t public
```

Hermes Message Pack

Hermes Message Pack is a tar gzipped file named .hmp